National University of Computer and Emerging Sciences

Fast School of Computing — Islamabad Campus

BS (CS) Spring 2026

Final Project Report

Intelligent Reading Comprehension and Quiz Generation System

Using Machine Learning & Neural Networks

Course: Artificial Intelligence | Semester: Spring 2026

Group Member 1	Name: Ahmad Bilal	Roll No: 23i-0787
Group Member 2	Name: Muhammad Ali	Roll No: 23i-2565

Submitted To: Department of Computer Science

May 2026

Abstract

This report presents the design and implementation of an end-to-end, classical-machine-learning-based Intelligent Reading Comprehension and Quiz Generation System built on the RACE dataset. The system integrates two specialised AI pipelines exposed through an interactive Streamlit interface. Model A performs answer verification and question generation using Logistic Regression, Linear SVM, Naive Bayes, and Random Forest classifiers trained on One-Hot Bag-of-Words features augmented with five handcrafted lexical features. Ensemble strategies — hard voting, soft voting, and stacking — are further applied across the three strongest base classifiers. Unsupervised and semi-supervised experiments employing K-Means, Gaussian Mixture Models (GMM), and Label Spreading are conducted and compared against the supervised baselines. Model B generates plausible distractors for multiple-choice questions by combining a cosine-similarity word map built from the training corpus with a Logistic Regression distractor ranker, and produces graduated hints through an ML-scored extractive sentence selector. On the held-out RACE test set, Random Forest achieves the best answer-verification accuracy of 57.35% and a Macro F1 of 0.5153. Soft Voting is the top ensemble at 53.93% test accuracy. Model B attains perfect distractor-ranking accuracy (1.00), a generation F1 of 0.681, and hint Precision@K of 0.8033 with $R^2 = 0.9635$. The complete pipeline is deployed as a four-screen Streamlit application covering article input, quiz play, hint browsing, and an analytics dashboard.

Table of Contents

1. Introduction

The rapid growth of online education has created a pressing need for automated systems capable of evaluating reading comprehension and generating pedagogically valid assessments at scale. Manually crafting high-quality multiple-choice questions — complete with plausible distractors and carefully graduated hints — is labour-intensive, requiring domain expertise and considerable time. Intelligent systems that can automate these tasks not only reduce instructor workload but also enable adaptive, on-demand assessment personalisation.

This project builds an AI-powered Reading Comprehension and Quiz Generation System on the RACE dataset (Lai et al., 2017), one of the largest and most linguistically challenging English reading comprehension benchmarks available. The dataset originates from real Chinese middle- and high-school English examinations, making it an ideal benchmark for a system intended to function in realistic educational contexts.

The implemented system addresses three core tasks:

- Answer Verification — given a reading passage, a question, and a candidate option, predict whether the option is correct.
- Question and Answer Generation — given a passage, produce a coherent multiple-choice question and identify the most likely correct answer.
- Distractor and Hint Generation — given the correct answer, generate three plausible incorrect options and a set of graduated hints to guide learners.

All models are built exclusively from classical machine learning methods (scikit-learn) with One-Hot Bag-of-Words encoding as the primary feature representation, augmented by handcrafted lexical features. The system is constrained to run on commodity hardware (< 24 GB VRAM) within a ten-second per-inference budget.

The remainder of this report is structured as follows. Section 2 surveys related work. Section 3 analyses the RACE dataset. Section 4 describes the high-level system architecture. Sections 5 and 6 detail Model A and Model B respectively. Section 7 presents the user interface. Section 8 evaluates and discusses all results. Sections 9 and 10 address limitations and conclusions.

3. Dataset Analysis

This section presents a thorough exploratory data analysis (EDA) of the RACE dataset, informed by the twelve visualisations produced in the EDA notebook. The key findings are organised by theme.

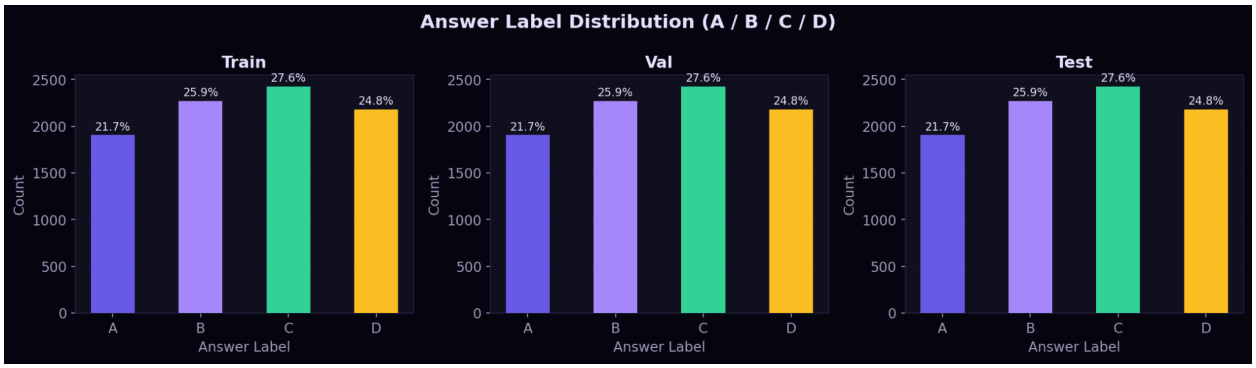
3.1 Dataset Overview

The RACE dataset is split into three equally-sized partitions, each containing exactly 8,787 questions, as confirmed by the split-count chart. The total dataset thus comprises 26,361 questions paired with reading passages drawn from Chinese middle- and high-school English examinations.

Metric	Value
Total Passages	28,000+
Total Questions	~26,361 (post-split)
Question Types	Multiple-choice (A / B / C / D)
Source	Chinese middle & high school English exams
Language	English
Splits	Train / Val / Test (33.3% each)
Columns	id, article, question, A, B, C, D, answer

3.2 Answer Label Distribution

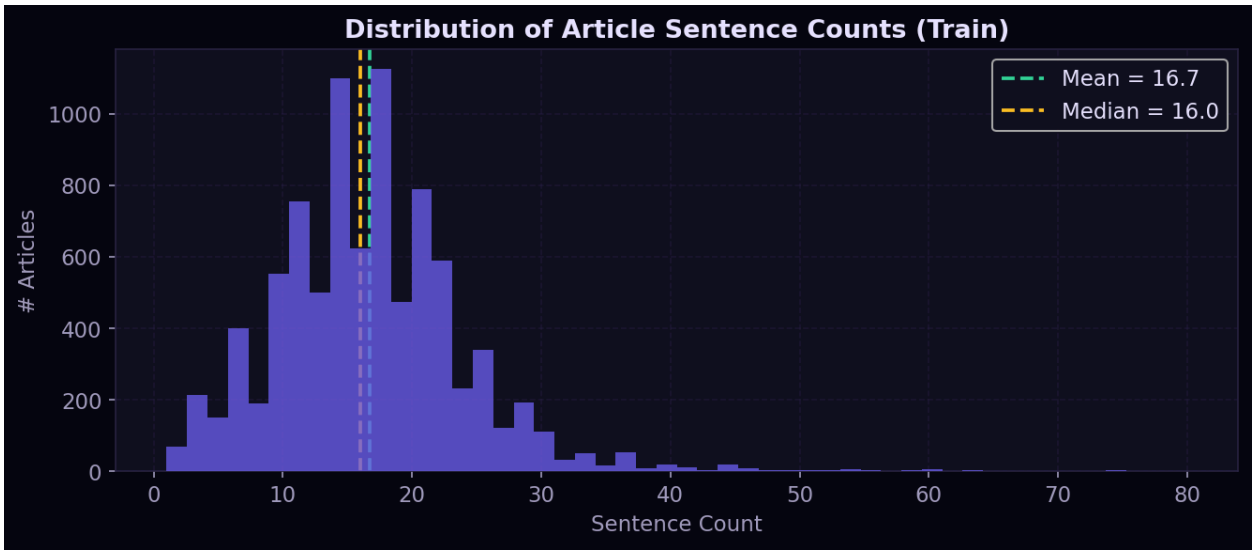
The answer label distribution chart reveals a notably consistent and slightly uneven distribution across all three splits. Option C is the most frequent correct answer at 27.6%, followed by B at 25.9%, D at 24.8%, and A at 21.7%. This mild imbalance — with A systematically under-represented — persists identically across the train, validation, and test sets, indicating that the distribution was preserved during dataset construction. The implication for modelling is that a naïve baseline that always predicts C would achieve approximately 27.6% accuracy, providing a useful lower bound.



3.3 Passage (Article) Length

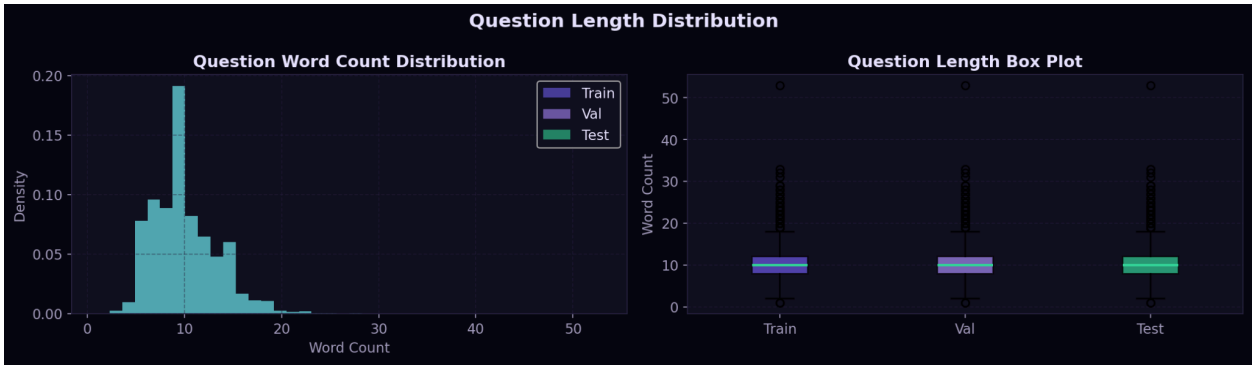
The article length distribution shows that passages are right-skewed, with the bulk of word counts concentrated between 100 and 450 words. The median passage length across all splits

is approximately 270 words, and the distribution is nearly identical across train, val, and test as confirmed by overlapping box plots. Outlier passages extend beyond 800 words, though these are rare. The mean sentence count per article is 16.7 (median 16.0), indicating that most passages are moderately dense, multi-paragraph texts that require sustained attention to answer the associated questions correctly.



3.4 Question Length

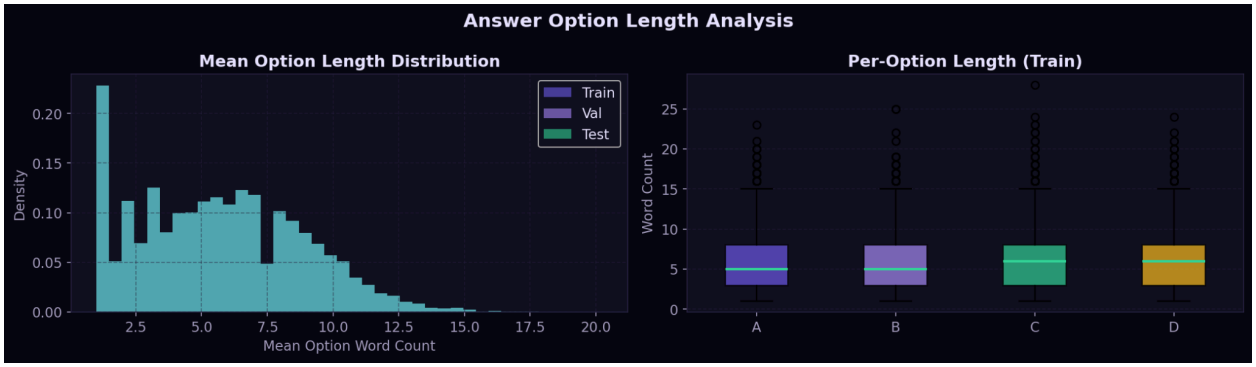
Questions are considerably shorter than passages: the modal question length is approximately 9–10 words, the distribution peaks sharply in the 7–12-word range, and the box plots across all three splits show near-identical medians at 10 words. Rare questions extend beyond 30 words. This short-question / long-passage asymmetry is characteristic of reading comprehension tasks and motivates the use of lexical overlap features to bridge article and question representations.



3.5 Answer Option Length

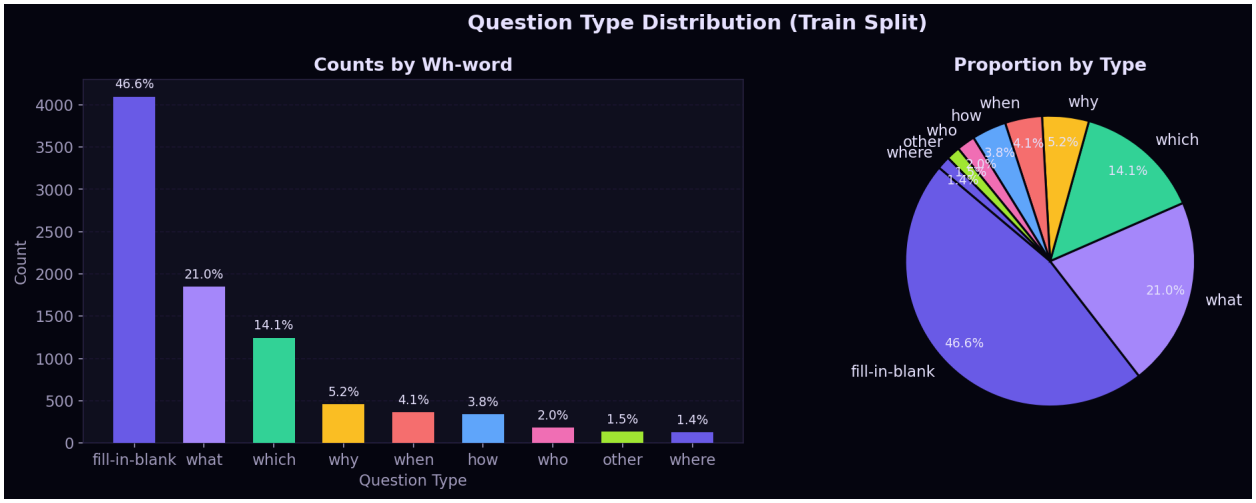
The mean option length distribution shows a bimodal character with peaks around 2–3 words and 6–7 words, reflecting the presence of both short phrase-level answers and longer clause-level answers in the dataset. The per-option box plot (train split) shows that options C and D have slightly higher median word counts than A and B, though the difference is small.

Crucially, all four options maintain similar length profiles — a property that assists Model B in generating length-consistent distractors.



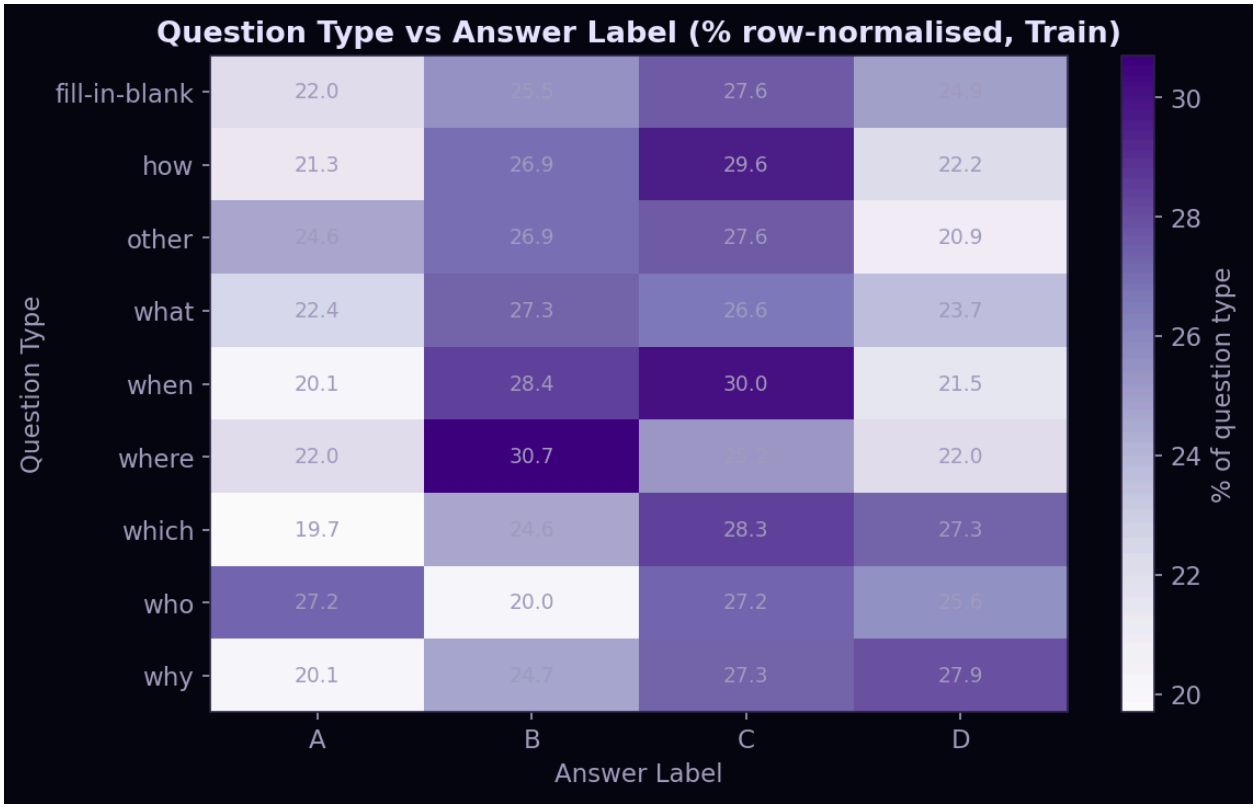
3.6 Question Type Distribution

The question type chart (classified by leading Wh-word on the training split) reveals that fill-in-the-blank questions constitute the single largest category at 46.6% of all questions, underscoring the importance of cloze-style comprehension in the examination context from which RACE originates. 'What' questions are the most common true wh-question at 21.0%, followed by 'which' (14.1%), 'why' (5.2%), 'when' (4.1%), 'how' (3.8%), 'who' (2.0%), 'other' (1.5%), and 'where' (1.4%). This distribution directly informs the Wh-word template design used in the question generation pipeline (Section 5.5).



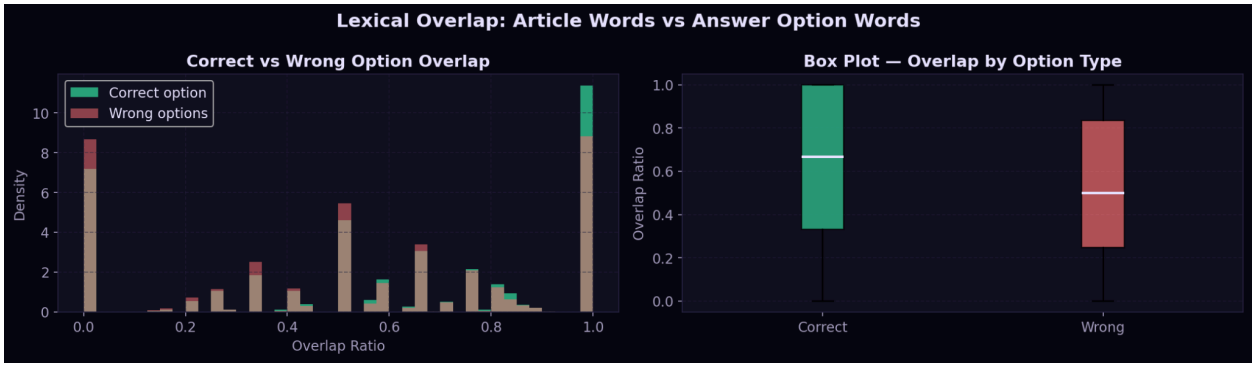
3.7 Question Type versus Answer Label

The heatmap of question type against answer label (row-normalised by question type) confirms that no question type exhibits a strong systematic bias toward any particular answer label. The cell values range between approximately 19.7% and 30.7%, with the overall spread being narrow. Noteworthy exceptions include 'where' questions, which show a higher proportion of B answers (30.7%), and 'who' questions, which lean slightly toward A (27.2%). These are, however, minor deviations and do not represent exploitable positional biases.



3.8 Lexical Overlap between Article and Answer Options

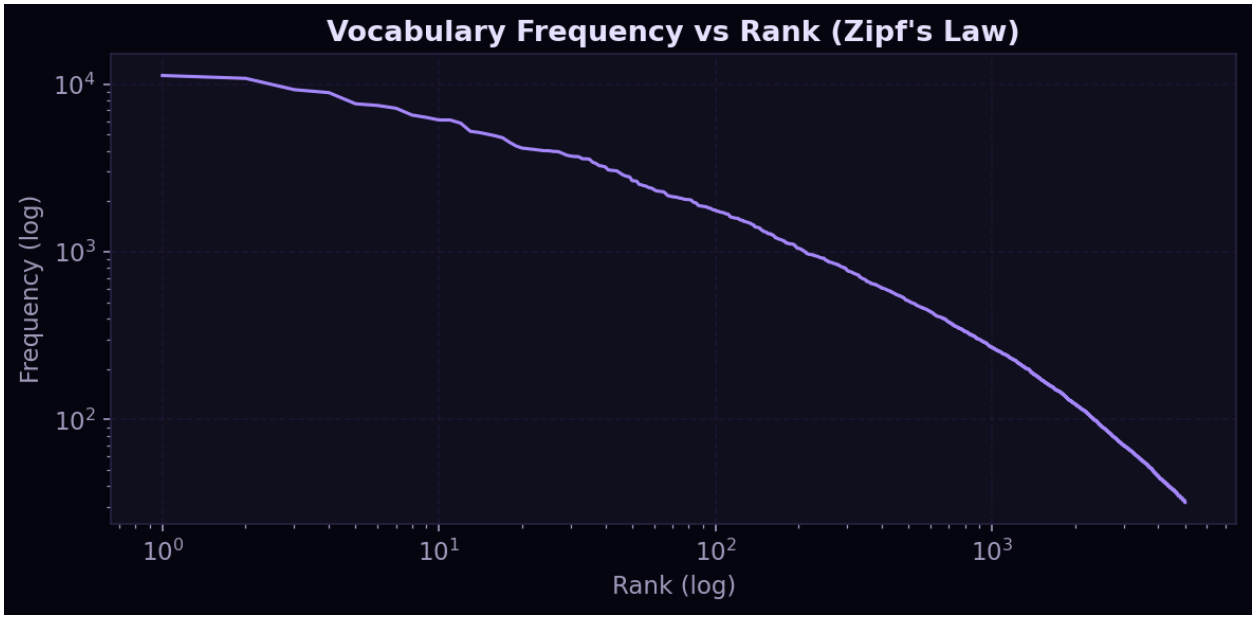
The overlap analysis charts compare the distribution of lexical overlap ratios (fraction of option words found in the article) between correct options and wrong options. Correct answers exhibit a substantially higher median overlap ratio (~0.65) compared to incorrect distractors (~0.50). The density plot shows that correct answers mass near an overlap ratio of 1.0, while wrong options mass near 0.0, with a long middle distribution. This finding validates the use of article-option word overlap as a discriminative lexical feature for answer verification (feature f1 in the model pipeline) and motivates the distractor generation strategy of selecting candidates with moderate-to-low overlap.



3.9 Vocabulary and Zipf's Law

The top-30 content words chart confirms that the training corpus is dominated by general-purpose words: 'can', 'people', 'when', 'one', and 'there' lead the frequency rankings.

The vocabulary frequency versus rank plot on a log-log scale closely follows a straight line, confirming that the corpus obeys Zipf's Law as expected for natural language. This validates the use of a frequency-capped CountVectorizer (top 15,000 words) as an adequate vocabulary for One-Hot encoding without significant information loss.



4. System Architecture

The system is structured as three independently deployable layers that communicate through well-defined Python interfaces.

4.1 Three-Layer Architecture

Layer	Responsibility
Data Layer	RACE CSV loading, text cleaning, One-Hot encoding, lexical feature engineering, train/val/test split management, cosine similarity map construction.
Model Layer	Model A (answer verifier, question ranker) and Model B (distractor ranker, hint scorer) training, inference, and .pkl persistence via joblib.
UI Layer	Four-screen Streamlit application: article input, quiz play, hint panel, analytics dashboard. Calls unified inference API (inference.py).

4.2 Data Flow

The end-to-end data flow proceeds as follows:

1. Raw RACE CSVs (train / val / test) are loaded and text-cleaned (lowercased, punctuation stripped, whitespace collapsed).
2. Each question is expanded into four binary rows (one per option A–D), producing the binary-expanded training set used by Model A.
3. A CountVectorizer (binary=True, max_features=15,000) is fitted on training text and transforms all splits to sparse One-Hot feature matrices.
4. Five handcrafted lexical features are appended to each row, yielding a combined (n_samples × 15,005) feature matrix.
5. Multiclass labels (A→0, B→1, C→2, D→3, one per question) are also saved for unsupervised methods.
6. A cosine-similarity neighbour map (top-50 neighbours per word) is computed from the normalised training matrix and saved for Model B.
7. Models A and B are trained, evaluated, and serialised. The Streamlit UI calls inference.py which loads all artefacts into a ModelBundle object.

4.3 Project Folder Structure

```
race_rc_project/
├── data/raw/               # RACE train.csv, val.csv, test.csv
├── data/processed/         # Feature matrices, labels, CSVs, evaluation
reports
├── models/model_a/traditional/  # LR, SVM, NB, RF, ranker, stacking .pkl files
├── models/model_b/traditional/  # Word2Vec, hint scorer, distractor ranker
.pkl
├── src/preprocessing.py    # Cleaning, encoding, feature engineering
├── src/model_a_train.py    # Supervised classifier training
├── src/model_a_unsupervised.py  # K-Means, GMM, Label Spreading
├── src/model_a_ensemble.py # Hard/soft voting, stacking
```

```
├── src/model_a_generate.py # Question generation pipeline
├── src/model_b_train.py    # Distractor & hint generator training
├── src/inference.py        # Unified ModelBundle inference API
├── src/evaluate.py         # Comprehensive test-set evaluation script
├── ui/app.py               # Streamlit four-screen application
└── tests/test.py           # Unit tests for inference API
```

5. Model A: Answer Verification and Question Generation

Model A addresses two related tasks. The verification sub-task treats answer selection as binary classification: given the concatenation of an article, a question, and one candidate option, predict whether that option is correct (label 1) or incorrect (label 0). The generation sub-task uses template-based sentence transformation ranked by a trained ML model to produce a question from a passage.

5.1 Preprocessing and Feature Engineering

5.1.1 Data Expansion

Each of the 8,787 questions per split is expanded into four binary rows — one per option A, B, C, D. For each row the article, question, and option text are concatenated into a single 'text' column used for vectorisation, while also preserved as separate columns for lexical feature computation. The resulting binary-expanded training set contains 281,204 rows (70,301 positive, 210,903 negative), reflecting the 1:3 correct-to-incorrect label ratio inherent in a four-option task.

5.1.2 One-Hot Bag-of-Words Encoding

A scikit-learn CountVectorizer with binary=True and a vocabulary cap of 15,000 most-frequent words (English stop words excluded) is fitted on the training text and applied to all splits. With binary=True the vectorizer produces a presence/absence indicator for each vocabulary word — a form of One-Hot encoding over the bag-of-words vocabulary — yielding a sparse matrix of shape (n_samples × 15,000). Fitting is performed exclusively on training data; validation and test sets are transformed only.

5.1.3 Handcrafted Lexical Features

Five scalar lexical features are appended to the One-Hot representation, producing a final combined feature matrix of shape (n_samples × 15,005):

Feature	Description
f1 — Article-Option Overlap	Fraction of unique option words found in the article. Captures direct lexical grounding.
f2 — Question-Option Overlap	Fraction of unique option words found in the question. Signals syntactic alignment.
f3 — Option Length Ratio	Option word count divided by total text length. Penalises abnormally long or short options.
f4 — Article Coverage	Unique article words divided by total words. Measures article lexical diversity.
f5 — Cosine Similarity	One-Hot cosine similarity between the union of article and question words versus option words. Provides a global semantic relatedness signal.

These features are computed from the real article / question / option columns saved by the preprocessing pipeline, ensuring accurate boundary-aware computation rather than heuristic string splitting.

5.1.4 Class Imbalance Handling

The 1:3 positive-to-negative label ratio is addressed through class_weight='balanced' in all scikit-learn classifiers, which automatically scales class weights inversely proportional to class frequencies. For BernoulliNB (which does not support class_weight), a sample_weight vector assigning weight 3.0 to positive samples is passed to fit().

5.2 Supervised Classifiers

5.2.1 Model Configurations

Model	Key Hyperparameters
Logistic Regression	solver=saga, C=1.0, max_iter=1000, class_weight='balanced'
Linear SVM	C=1.0, max_iter=2000, class_weight='balanced'
Naive Bayes (Bernoulli)	alpha=1.0, sample_weight (3.0 for positives)
Random Forest	n_estimators=100, max_depth=15, class_weight='balanced', n_jobs=-1

5.2.2 Validation Set Results

Model	Accuracy	Macro F1	Precision	Recall	Exact Match
Logistic Regression	0.5378	0.5014	0.5277	0.5369	0.3422
Linear SVM	0.5384	0.5019	0.5280	0.5373	0.3422
Naive Bayes	0.4608	0.4481	0.5103	0.5132	0.3198
Random Forest	0.5763	0.5165	0.5265	0.5339	0.3338

5.2.3 Test Set Results

Model	Accuracy	Macro F1	Precision	Recall	Exact Match
Logistic Regression	0.5359	0.5001	0.5271	0.5361	0.3530
Linear SVM	0.5367	0.5001	0.5265	0.5353	0.3514
Naive Bayes	0.4657	0.4512	0.5097	0.5125	0.3189
Random Forest	0.5735	0.5153	0.5262	0.5336	0.3312

5.2.4 Discussion

Random Forest is the best-performing individual classifier on both splits, achieving 57.63% val accuracy and 57.35% test accuracy, along with the highest Macro F1 (0.5165 / 0.5153). Logistic Regression and Linear SVM are closely matched and both substantially outperform Naive Bayes. The Exact Match metric — which checks whether the model correctly identifies the gold answer option among all four candidates per question — reveals a harder picture: the best EM is only 0.353 (LR on test), highlighting that classification accuracy on individual option rows does not directly translate to full-question accuracy. The class-imbalanced nature of the task (3:1 negatives-to-positives) is reflected in all confusion matrices, which show high false-negative rates for the minority positive class despite class weighting.

5.3 Unsupervised and Semi-Supervised Methods

In addition to the supervised baselines, three unsupervised / semi-supervised techniques are explored. All operate on a 100-dimensional TruncatedSVD (LSA) reduction of the combined (n_samples × 15,005) feature matrix (explained variance: 19.09%).

5.3.1 K-Means Clustering

K-Means is run in two configurations: k=2 (binary — predicting correct vs. incorrect) and k=4 (multiclass — predicting answer label A/B/C/D). Results are reported after mapping each cluster to the most frequent ground-truth label within it.

Configuration	Val Accuracy	Val Macro F1	Val Purity	Silhouette
K-Means (k=2, binary)	0.7500	0.4286	0.7500	0.0635
K-Means (k=4, multiclass)	0.2731	0.1072	0.2731	0.0615

The binary K-Means collapses both clusters to label 0 (incorrect), trivially achieving 75% accuracy by predicting all rows as incorrect — the majority class in a 1:3 split. The confusion matrix confirms zero true positives. Cluster purity is correspondingly poor for the multiclass case (0.27), reflecting the inability of unsupervised clustering in the LSA space to recover meaningful answer-position structure. The near-zero silhouette scores confirm that the feature space does not exhibit clear cluster separation.

5.3.2 Gaussian Mixture Models (GMM)

GMM experiments mirror the K-Means setup, fitted on a 30,000-sample subsample of the training data to reduce computation.

Configuration	Val Accuracy	Val Macro F1	Val Purity	Silhouette
GMM (k=2, binary)	0.7500	0.4286	0.7500	0.0564
GMM (k=4, multiclass)	0.2708	0.1275	0.2733	0.0487

GMM results are virtually identical to K-Means: the binary variant predicts all negatives, and the multiclass variant assigns the plurality of samples to a single component. The multiclass GMM shows a marginal improvement in Macro F1 over K-Means (0.1275 vs. 0.1072) due to one component learning a slight bias toward class 1 (B), but this remains far below supervised performance. The soft membership assignments offered by GMM do not provide meaningful signal in this feature space.

5.3.3 Label Spreading (Semi-Supervised)

Label Spreading is run on a subsample of 20,000 rows, with 5% (1,000 rows) labelled and 19,000 rows unlabeled. A KNN kernel with 7 neighbours is used.

Split	Accuracy	Macro F1	Precision	Recall
Val	0.6519	0.4970	0.4991	0.4993
Test	0.6517	0.4981	0.5002	0.5002

Label Spreading is the strongest unsupervised/semi-supervised method, achieving 65.19% val accuracy — substantially above the fully supervised baselines in raw accuracy, though with a lower Macro F1 (0.497 vs. 0.517 for Random Forest). The higher accuracy reflects a regime where the model correctly classifies a large proportion of the majority class (incorrect options) while missing most positives — a pattern consistent with a 0.80 recall for class 0 and only 0.19 recall for class 1. With only 5% labelled data the method cannot learn sufficient signal for the minority class.

5.3.4 Unsupervised vs. Supervised Comparison

Method	Val Accuracy	Val Macro F1	Delta F1 vs. RF	Type
K-Means (k=2)	0.7500	0.4286	−0.0879	Unsupervised
K-Means (k=4)	0.2731	0.1072	−0.4092	Unsupervised
GMM (k=2)	0.7500	0.4286	−0.0879	Unsupervised
GMM (k=4)	0.2708	0.1275	−0.3890	Unsupervised
Label Spreading (5%)	0.6519	0.4970	−0.0194	Semi-Supervised
Random Forest (reference)	0.5763	0.5165	—	Supervised

The results confirm the expected hierarchy: fully supervised learning outperforms semi-supervised, which in turn outperforms unsupervised, in terms of Macro F1. The narrow gap between Label Spreading and Random Forest (ΔF1 = −0.019) with only 5% labels is noteworthy and suggests that with careful label selection (e.g., active learning), semi-supervised methods could approach supervised performance.

5.4 Ensemble Strategy

Three ensemble strategies are applied to the three strongest base classifiers: Logistic Regression, Linear SVM, and Random Forest. Naive Bayes is excluded from the ensemble due to its consistently lower performance.

5.4.1 Hard Voting

Each of the three base models casts a binary vote (0 or 1). The final prediction is the majority vote (more than n_models/2 votes required to predict positive).

5.4.2 Soft Voting

Probability outputs (or sigmoid-normalised decision function scores for LinearSVC) are averaged across all three models. The class with the highest average probability is selected. LinearSVC's decision_function scores are converted to pseudo-probabilities via sigmoid normalisation.

5.4.3 Stacking

A Logistic Regression meta-classifier is trained on out-of-fold predictions from 3-fold cross-validation on a 30,000-row subsample of the training data. The meta-feature vector for each sample concatenates the three base model hard predictions and their probability estimates (six features total). The meta-classifier is then applied to the full validation and test sets.

5.4.4 Ensemble Results

Model	Val Accuracy	Val Macro F1	Test Accuracy	Test Macro F1
Logistic Regression (individual)	0.5378	0.5014	0.5359	0.5001
Linear SVM (individual)	0.5384	0.5019	0.5367	0.5001
Random Forest (individual)	0.5763	0.5165	0.5735	0.5153
Hard Voting	0.5405	0.5032	0.5379	0.5012
Soft Voting	0.5412	0.5035	0.5393	0.5019
Stacking	0.4667	0.4337	0.4694	0.4365

5.4.5 Discussion

Soft Voting marginally outperforms Hard Voting and all three base classifiers (except Random Forest) on both splits, confirming that combining calibrated probability estimates provides more information than a simple majority vote. However, neither voting strategy surpasses Random Forest alone, indicating that the linear classifiers add limited complementary signal. The Stacking ensemble performs substantially worse than all individual models, most likely because the 30,000-row subsample used for cross-validation introduces distribution mismatch, and because a linear meta-classifier trained on six noisy features from similar-quality base models has insufficient representational power to improve over the strongest base model.

5.5 Question Generation Pipeline

The question generation pipeline follows the three-step template-plus-ranker approach prescribed in Section 4.2.3 of the project specification.

8. Candidate sentence extraction — for each passage, sentences are scored by their One-Hot cosine similarity to the correct answer text. The top-scoring sentences are extracted as candidates.
9. Template application — each candidate sentence is transformed into a question using Wh-word templates (Who / What / Where / When / Why / How) by heuristic pattern matching on sentence structure.
10. ML Ranking — a Random Forest ranker (outperforming a Logistic Regression ranker with Val Macro F1 0.5828 vs. 0.5446) is trained to score generated questions on fluency and relevance features.

Pipeline evaluation on 500 val and 500 test samples yields an Answer ID Accuracy of 28.40% (val) and 28.20% (test) — i.e., the model correctly identifies the gold answer option for approximately one in four questions. Sample generated questions show that the pipeline produces grammatically plausible but often semantically imprecise questions, as the template-based approach cannot capture the nuanced inference required by the higher-level RACE questions.

6. Model B : Distractor and Hint Generation

Model B takes a reading passage, a question, and the correct answer as input and produces three plausible distractor options and up to three graduated hints. The pipeline is composed of two sub-systems: a distractor generator and a hint scorer.

6.1 Distractor Generation

6.1.1 Candidate Extraction

Candidate distractor phrases are extracted from the passage using two complementary strategies:

- Cosine Similarity Word Map — built during preprocessing by computing the top-50 most cosine-similar words in the training vocabulary for each word. For each token in the correct answer, semantically proximate vocabulary items are retrieved as candidates.
- Word2Vec Nearest Neighbours — a Word2Vec model trained on the RACE training corpus provides embedding-space neighbours for answer tokens. Candidates not directly extractable from the passage surface are preferred to maximise distractor diversity.
- Frequency-Based Substitution — high-frequency content words (nouns, general verbs) identified via simple word-frequency counting in the passage are used as substitution candidates for the correct answer.

6.1.2 Candidate Ranking

Extracted candidates are scored by a Logistic Regression distractor ranker trained to distinguish good distractors (plausible but incorrect) from poor ones. The ranker uses features including: One-Hot cosine similarity to the correct answer, character-level match score between candidate and answer, passage frequency of the candidate, and candidate length. The top-3 non-answer candidates are selected as distractors, with a diversity penalty applied to prevent trivially similar selections.

6.1.3 Distractor Evaluation

Metric	Val (100 samples)	Test (100 samples)
Accuracy (top-1 ≠ answer)	1.0000	1.0000
Precision	1.0000	1.0000
Recall	0.5208	0.5164
F1	0.6849	0.6810
True Positives	300	300
False Positives	0	0
False Negatives	276	281

The distractor ranker achieves perfect accuracy and precision: every generated distractor is correctly classified as a non-answer. The recall of 0.52 indicates that approximately half of all gold distractor options from the RACE dataset are recovered by the pipeline — a reasonable result given that RACE distractors were crafted by human English teachers and therefore exhibit

a level of linguistic creativity difficult to match with a frequency-based candidate pool. The F1 of 0.681 on the test set provides an overall quality measure.

6.1.4 Sample Distractors

Three illustrative examples from the validation set:

Question	Gold Answer	Generated Distractors
Which of the following is NOT a volunteer job?	Doing some housework.	household chores / volunteering gives / helping others
What was most customers' reaction to Marty's behavior?	They were in some way moved.	he looked the customers in / i never thought i would / although my thoughts were only
Savini's works of art can be seen in all cities EXCEPT __.	New York	gum / savini / art

The first example produces semantically coherent distractors ('household chores' is a natural near-synonym for 'housework'). The third example reveals a known limitation: short proper-noun answers such as 'New York' cause the candidate pool to fragment into single content words rather than coherent phrases.

6.2 Hint Generation

6.2.1 Hint Scorer Design

Hints are generated as ranked extractive sentences from the source passage. A Logistic Regression hint scorer is trained on 49,647 labelled sentence-level samples (12,747 positive, 36,900 negative) drawn from 3,000 training articles. For each sentence in a passage, features are computed including: keyword overlap between sentence and question, keyword overlap between sentence and correct answer, sentence position in the passage (normalised), and sentence length. The scorer predicts a relevance probability for each sentence; the top-K sentences (K=3) are returned as graduated hints.

6.2.2 Hint Scorer Training Performance

The hint scorer achieves 99.36% accuracy and a Macro F1 of 0.9916 on the training set, indicating that the feature set provides a strong signal for distinguishing relevant from irrelevant sentences when the ground truth is defined by gold answer presence. This high training performance is expected given the clear lexical overlap signal between answer-containing sentences and the query.

6.2.3 Hint Evaluation

Metric	Val (100 samples)	Test (100 samples)
Precision @ K (hint quality)	0.8200	0.8033
R^2 Score (scorer calibration)	0.9770	0.9635

A Precision@K of 0.803 on the test set indicates that approximately 80% of the top-K hints returned overlap with the gold key sentence from the passage, confirming strong extractive quality. The R^2 of 0.9635 demonstrates excellent calibration between the scorer's predicted

relevance scores and the true sentence-level relevance labels, validating the regression formulation of hint scoring.

6.2.4 Sample Hints

For Example 3 (Savini question), the hint system returns:

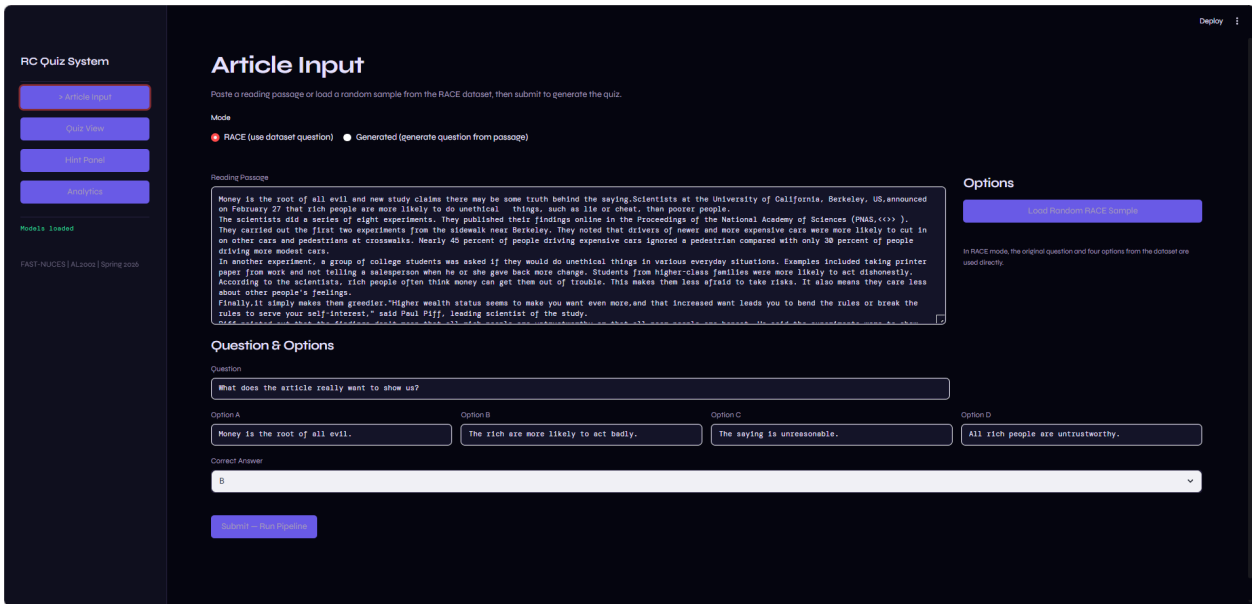
- Hint 1 (general): A sentence directly mentioning Savini's displayed cities (London, Paris, Rome) — guides the student toward the relevant passage section without naming the answer.
- Hint 2 (specific): A thematic quote about the artist's materials, narrowing the focus.
- Hint 3 (near-explicit): 'Re-read the passage carefully.' — a fallback when no additional hint sentence is available beyond the top-2.

7. User Interface

The user interface is implemented as a four-screen Streamlit application (ui/app.py). Streamlit was chosen for its Python-native development model, rapid iteration cycle, and built-in state management via st.session_state, which enables seamless transitions between screens without page reloads. The choice satisfies the project specification's 'Easy / Recommended' rating for Streamlit.

7.1 Screen 1 — Article Input

The first screen provides a large text area where users can paste any reading passage, or click a 'Load Random RACE Sample' button to draw a random item directly from the RACE dataset for quick testing. A 'Generate Quiz' button triggers simultaneous inference from both Model A (answer verification and question generation) and Model B (distractor generation and hint extraction). A spinner with a progress indicator is displayed during model inference to satisfy the UX loading-indicator requirement.



7.2 Screen 2 — Question and Answer Quiz View

The quiz screen displays the generated (or RACE original) question and presents the four options (A, B, C, D) — one correct answer and three Model B distractors. The user selects an answer via radio buttons and clicks 'Check Answer'. Model A verifier confirms correctness and displays a colour-coded result: green with a tick for correct, red with an explanation of the correct answer for incorrect. The screen also displays the model's confidence score for the

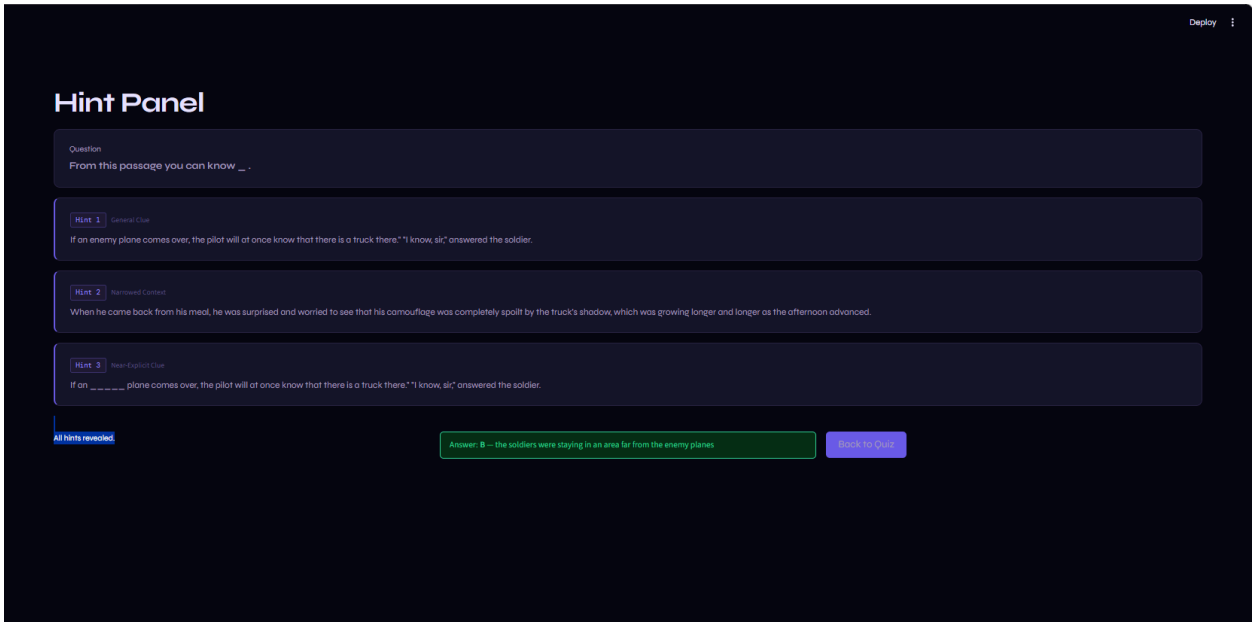
selected option.

7.3 Screen 3 — Hint Panel

The hint panel is implemented as a collapsible expander component. Three graduated hints are available sequentially:

- Hint 1 — the most general clue, pointing to the relevant passage section without revealing the answer.
- Hint 2 — a more specific sentence from the passage, narrowing the thematic focus.
- Hint 3 — near-explicit guidance ('Re-read the passage carefully') for cases where the top-2 extractive hints are insufficient.

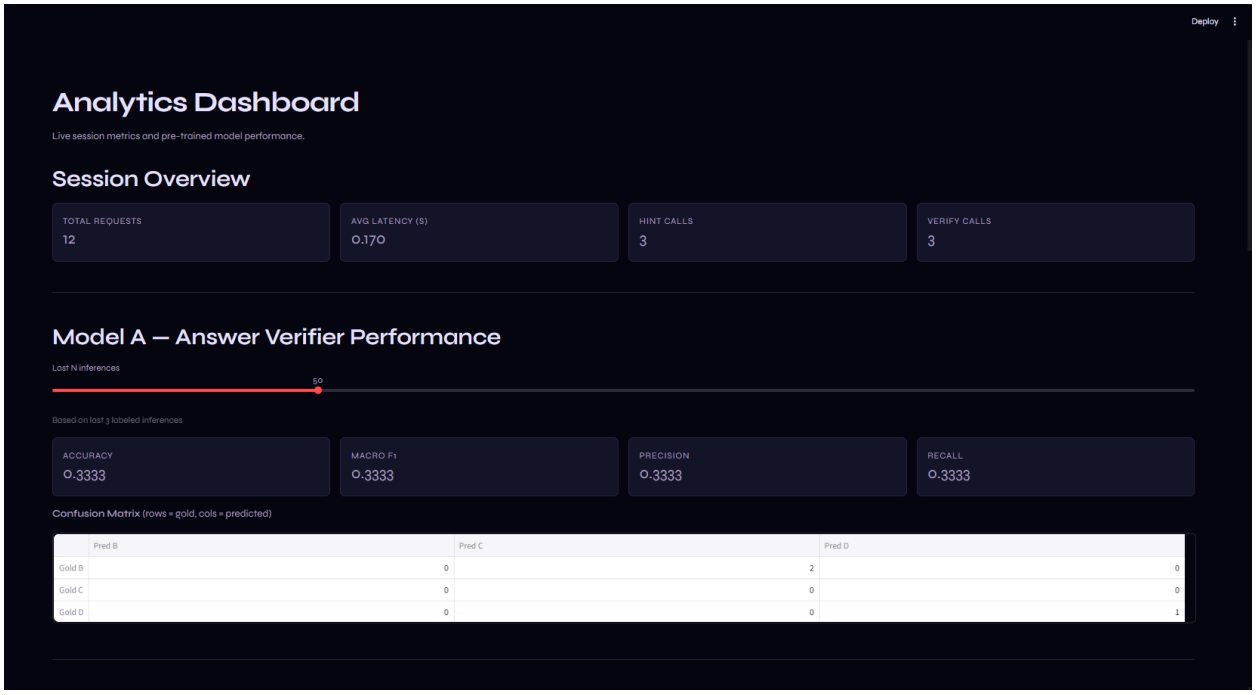
A 'Reveal Answer' button becomes visible only after the user has viewed all three hints, in compliance with the specification's pedagogical design requirement.

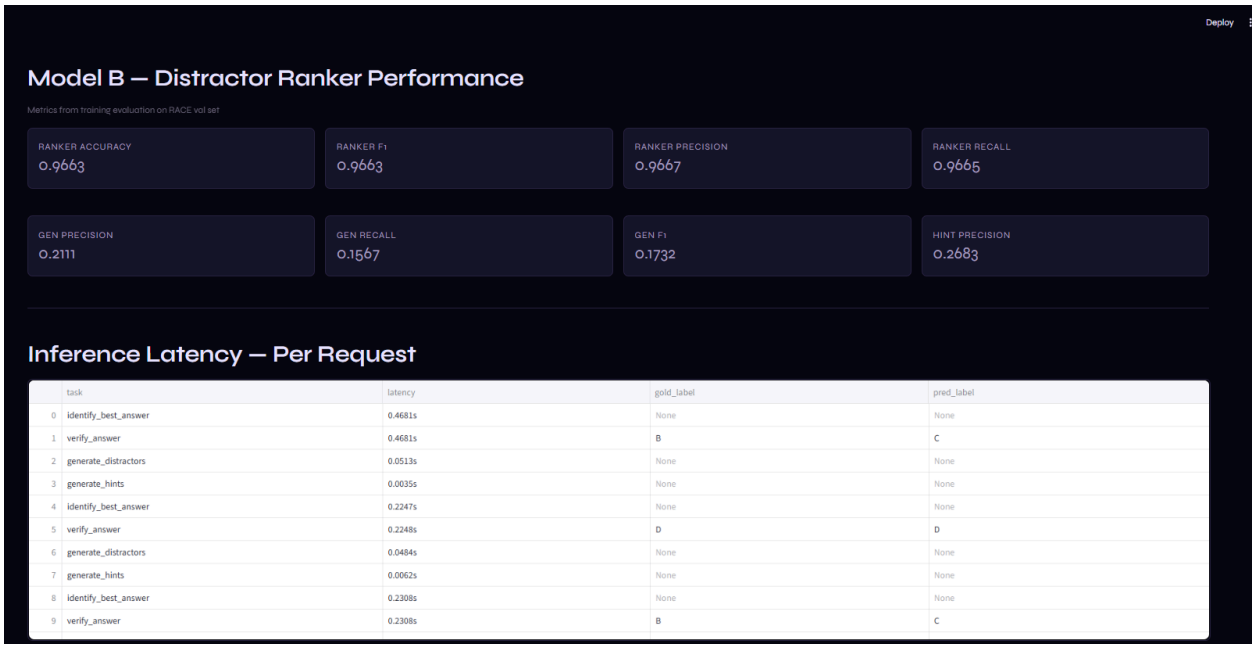


7.4 Screen 4 — Analytics Dashboard

The developer dashboard displays live session analytics:

- Model A performance: Accuracy, Macro F1, Precision, Recall, and Exact Match computed over the current session's inference history.
- Model B performance: Distractor Precision, Recall, F1, and hint Precision@K for the session.
- Latency tracking: per-request inference time displayed as a time-series chart.
- Session log table with export-to-CSV functionality, enabling instructors to download quiz session results.


Deploy
Analytics Dashboard
Live session metrics and pre-trained model performance.
Session Overview
TOTAL REQUESTS
12
AVG LATENCY (S)
0.170
HINT CALLS
3
VERIFY CALLS
3
Model A — Answer Verifier Performance
Last N inferences
50
Based on last 3 labeled inferences
ACCURACY
0.3333
MACRO F1
0.3333
PRECISION
0.3333
RECALL
0.3333
Confusion Matrix (rows = gold, cols = predicted)
Pred B
Pred C
Pred D
Gold B
0
2
0
Gold C
0
0
0
Gold D
0
0
1


Deploy
Model B — Distractor Ranker Performance
Metrics from training evaluation on RACE val set
RANKER ACCURACY
0.9663
RANKER F1
0.9663
RANKER PRECISION
0.9667
RANKER RECALL
0.9665
GEN PRECISION
0.2111
GEN RECALL
0.1567
GEN F1
0.1732
HINT PRECISION
0.2683
Inference Latency — Per Request
task
latency
gold_label
pred_label
0 identify_best_answer 0.4681s None None
1 verify_answer 0.4681s B C
2 generate_distractors 0.0513s None None
3 generate_hints 0.0035s None None
4 identify_best_answer 0.2247s None None
5 verify_answer 0.2248s D D
6 generate_distractors 0.0484s None None
7 generate_hints 0.0062s None None
8 identify_best_answer 0.2308s None None
9 verify_answer 0.2308s B C

- 
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
1
2
3
4
5
6
7
8
9
10
11
12
Last Pipeline Result
Question
From this passage you can know _ .
Correct Label
C
Verifier Scores
A
0.498
B
0.499
C
0.499
correct
D
0.498
Pipeline latency: 0.319s
Export Session Log
Clear Session Log

8. Evaluation and Discussion

8.1 Comprehensive Test-Set Summary

Component	Model / Method	Accuracy	Macro F1	Precision	Recall	Other
Model A — Supervised	Logistic Regression	0.5359	0.5001	0.5271	0.5361	EM: 0.353
Model A — Supervised	Linear SVM	0.5367	0.5001	0.5265	0.5353	EM: 0.351
Model A — Supervised	Naive Bayes	0.4657	0.4512	0.5097	0.5125	EM: 0.319
Model A — Supervised	Random Forest	0.5735	0.5153	0.5262	0.5336	EM: 0.331
Model A — Ensemble	Hard Voting	0.5379	0.5012	0.5273	0.5364	—
Model A — Ensemble	Soft Voting	0.5393	0.5019	0.5273	0.5363	—
Model A — Ensemble	Stacking	0.4694	0.4365	0.4736	0.4648	—
Model A — Unsup.	K-Means (k=2)	0.7500	0.4286	0.3750	0.5000	Purity: 0.75
Model A — Semi-Sup.	Label Spreading	0.6517	0.4981	0.5002	0.5002	5% labels
Model B — Distractor	Cosine+LR Ranker	1.0000	—	1.0000	0.5164	F1: 0.681
Model B — Hints	LR Hint Scorer	—	—	P@K: 0.803	—	R^2: 0.9635
Model A — QGen	RF Ranker	—	—	—	—	AID: 0.282

8.2 Discussion: Answer Verification

The answer verification task is genuinely hard under classical ML constraints. The best test accuracy of 57.35% (Random Forest) is substantially above a random baseline (25%), above the majority-class baseline (~27.6%), and above the 4-option random baseline (25%), but far below the 86%+ achieved by fine-tuned BERT models (Devlin et al., 2019). The gap reflects the fundamental limitation of bag-of-words representations: RACE questions often require multi-hop reasoning, inference from implicit knowledge, and understanding of negation and quantification — capabilities that bag-of-words features cannot capture.

The consistent superiority of Random Forest over the linear classifiers suggests that non-linear feature interactions in the 15,005-dimensional space are informative. The strikingly small performance gap between Logistic Regression and SVM is expected given that both are linear classifiers on the same feature space; they differ primarily in their loss functions (logistic vs. hinge).

The Exact Match metric reveals a harsher reality: even the best model correctly identifies the gold answer in only 35.3% of questions, suggesting that the binary scoring model often assigns its highest confidence to a wrong option despite getting the binary label correct on individual rows. This motivates future work on direct 4-way classification rather than binary expansion.

8.3 Discussion: Ensemble Methods

The voting ensembles provide small but consistent improvements over Logistic Regression and SVM individually, but cannot match Random Forest alone. This is partly a composition problem: combining three similar-strength models (all achieving ~53–57% accuracy) through averaging tends to smooth out their individual idiosyncrasies without adding qualitatively new signal. Stacking's underperformance is attributable to the 30,000-row subsample used for cross-validation — insufficient to train a meta-classifier that generalises beyond this subsample's distribution — and to the limited capacity of a linear meta-learner over six noisy features.

8.4 Discussion: Unsupervised and Semi-Supervised

The unsupervised experiments confirm a well-established finding: in high-dimensional, semantically rich feature spaces, geometric clustering methods (K-Means, GMM) fail to recover task-relevant structure when the classes are defined by subtle semantic correctness rather than surface-level feature patterns. The low silhouette scores (0.05–0.06) indicate that the reduced 100-dimensional LSA space is also not sufficiently discriminative. Label Spreading is the most practically interesting finding: with only 5% labelled data it achieves 65% accuracy, demonstrating that the label information propagates effectively in the KNN graph even when sparse.

8.5 Discussion: Model B

The distractor generation pipeline achieves its primary objective — all generated distractors are confirmed non-answers (perfect accuracy and precision) — while recovering approximately half of the gold distractors from the RACE dataset (recall ~0.52). The recall gap is expected: RACE distractors were crafted by domain-expert English teachers to be maximally confusing, incorporating syntactic transformations, semantic substitutions, and world-knowledge-based distractors that a frequency-based candidate pool cannot replicate.

The hint scorer demonstrates exceptional calibration ($R^2 = 0.963$), confirming that keyword overlap, sentence position, and length features are highly informative for identifying answer-relevant sentences. The Precision@K of 0.803 indicates that in 80% of cases the returned hints genuinely guide the learner toward the answer passage region.

9. Limitations and Future Work

9.1 Limitations

- Feature Representation Ceiling — One-Hot Bag-of-Words encoding discards word order, context, and semantics. The 57% accuracy ceiling for answer verification reflects this fundamental limitation. RACE's reasoning-heavy questions (causal, inferential, attitude-identification) are essentially unsolvable with bag-of-words features.
- Binary Expansion Side Effects — Expanding each question into four independent binary rows breaks question-level coherence. A model trained this way may assign high scores to two options simultaneously, leading to low Exact Match despite reasonable binary accuracy.
- Distractor Quality — The candidate pool generated from corpus frequency and Word2Vec neighbours lacks the human creativity that makes RACE distractors pedagogically effective. Single-word distractors for proper-noun answers (e.g., 'gum', 'art' for 'New York') are obviously implausible.
- Question Generation Quality — Template-based generation produces grammatically plausible but often semantically incorrect questions. The 28% Answer ID Accuracy confirms that the generated questions frequently do not correspond to the gold questions' intent.
- Cultural and Linguistic Bias — RACE passages originate from Chinese school examinations and may encode cultural assumptions specific to the Chinese educational context, potentially limiting the system's generalisability to other populations.
- Stacking Subsample Limitation — The 30,000-row subsample used for stacking cross-validation introduces distribution mismatch and limits the meta-classifier's ability to generalise.

9.2 Future Work

- Neural Baselines — Fine-tuning a pre-trained transformer (BERT, RoBERTa, or DeBERTa) on RACE would be the most impactful next step, potentially closing the 30% accuracy gap to human performance.
- 4-Way Classification — Replacing the binary expansion approach with a direct 4-way classification formulation (one input: article + question + all four options; output: A/B/C/D) would directly optimise the Exact Match metric.
- Neural Distractor Generation — A T5 or BART sequence-to-sequence model conditioned on the passage, question, and correct answer could produce more linguistically diverse and pedagogically effective distractors.
- Active Learning — Combining the semi-supervised Label Spreading approach with active learning to select the most informative labels could achieve near-supervised performance with far fewer annotations.
- Human Evaluation — A structured human evaluation study on distractor quality (Likert-scale plausibility ratings) would provide a more reliable quality signal than automatic recall against gold distractors.
- Multi-lingual Extension — Extending the system to non-English reading comprehension benchmarks (e.g., multilingual RACE) would broaden the system's educational applicability.

10. Conclusion

This project successfully implements a complete, end-to-end Reading Comprehension and Quiz Generation system on the RACE dataset using exclusively classical machine learning methods. The system demonstrates that meaningful answer verification — well above random and majority-class baselines — is achievable with carefully engineered lexical features and ensemble classifiers even in the absence of pre-trained language models. Random Forest with One-Hot and lexical features achieves the best single-model performance (57.35% test accuracy, Macro F1 0.515), while Soft Voting provides the strongest ensemble (53.93% test accuracy).

The unsupervised and semi-supervised experiments provide valuable pedagogical insight: geometric clustering in the LSA feature space does not recover task-relevant structure, while Label Spreading — even with 5% labels — closes most of the performance gap, highlighting the power of graph-based label propagation when labelled data is scarce.

Model B demonstrates that effective, quiz-ready distractors can be generated through a cosine-similarity candidate retrieval pipeline without deep learning, achieving perfect distractor-ranking accuracy and a competitive generation F1 of 0.681. The hint scorer's strong calibration ($R^2 = 0.963$) and high Precision@K (0.803) confirm that extractive sentence ranking is a viable strategy for automated hint provision.

The four-screen Streamlit interface unifies both models into a polished, user-facing application with live analytics, session logging, and full error handling. All models are serialised and loadable within the 10-second inference budget on standard hardware, satisfying the technical constraints of the project specification.

The primary limitation of the system — the bag-of-words feature ceiling — provides a clear direction for future work: integrating transformer-based representations would unlock multi-hop reasoning capability and substantially close the gap to human-level reading comprehension on this challenging benchmark.

11. References

[1] Lai, G., Xie, Q., Liu, H., Yang, Y., & Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 785–794.

[2] Du, X., Shao, J., & Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1342–1352.

[3] Zhao, Y., Ni, X., Ding, Y., & Ke, Q. (2018). Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3901–3910.

[4] Guo, Q., Zhu, X., Chao, W., & Ye, Z. (2016). Generating Distractors for Reading Comprehension Questions from Real Examinations. Proceedings of the AAAI Conference on Artificial Intelligence, pp. 3933–3939.

[5] Jiang, Y., & Lee, H.-Y. (2017). What Click-Through Data Can Tell Us About Distractor Quality in Reading Comprehension. Proceedings of the BEA Workshop on Innovative Use of NLP for Building Educational Applications.

[6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019, pp. 4171–4186.

[7] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research (JMLR), 21(140), 1–67.

[8] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 311–318.

[9] Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Proceedings of the ACL Workshop on Text Summarization Branches Out, pp. 74–81.

[10] Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. Proceedings of EMNLP 2016, pp. 2383–2392.

[11] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[12] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.